CSC 2105: Data Structure

Array & String

Fahad Ahmed
Assistant Professor, Department of Computer Science
Faculty of Science & Information Technology
American International University-Bangladesh (AIUB)
fahad.ahmed@aiub.edu

DATA STORAGE CONCEPT

We have already gone through how computers represent information using binary number system.

- But we need to understand how we are going to representation information in our programming, which we will use as the basis for our computational processes.
- To do this, we need to decide on representations for *numeric values* and for *symbolic ones*.
- We will use the concept of variables.

VARIABLES

- Let us think that, I ask you to retain the number 5 in your mental memory, and then I ask you to memorize also the number 2 at the same time. You have just stored two different values (5, 2) in your memory. Now, if I ask you to add 1 to the first number I said, you should be retaining the numbers 6 (that is 5+1) and 2 in your memory. Now subtract the second value from the first and you obtain 4 as result.
- The whole process that you have just done with your mental memory is a simile of what a computer can do with two variables. The same process can be expressed in C/C++ with the following instruction set:

```
1. a = 5
2. b = 2
3. a = a + 1
4. result = a - b
```

VARIABLES

```
1. a = 5
2. b = 2
3. a = a + 1
4. result = a - b
```

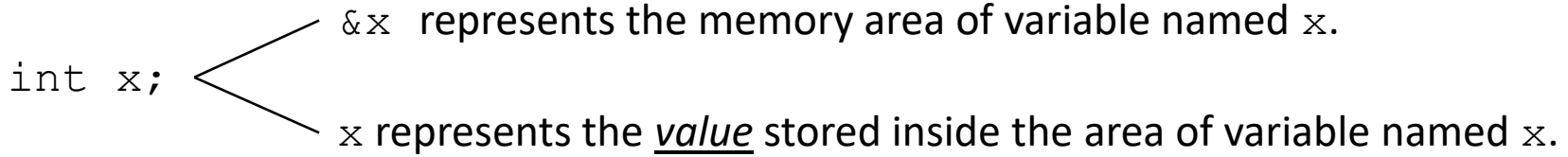
- Obviously, this is a very simple example since we have only used two small integer values, but consider that your computer can store millions of numbers like these at the same time and conduct sophisticated mathematical operations with them.
- Therefore, we can define a *variable* as a portion of memory to store a determined value. Each variable needs an identifier that distinguishes it from the others. For example, in the previous code the *variable identifiers* were `a`, `b` and `result`, but we could have called the variables any names we wanted to invent, as long as they were valid identifiers.

VARIABLES

- So, a variable is a *storage location* and an associated *symbolic name* (an *identifier*) which contains some known or unknown quantity or information, a value.
- The variable name is the usual way to reference the stored value; this separation of name and content allows the name to be used independently of the exact information it represents.
- The identifier in computer source code can be bound to a value during run time, and the value of the variable may thus change during the course of program execution.
- In computing, a variable may be employed in a repetitive process: assigned a value in one place, then used elsewhere, then reassigned a new value and used again in the same way.
- Compilers have to replace variables' symbolic names with the actual locations of the data in the memory.

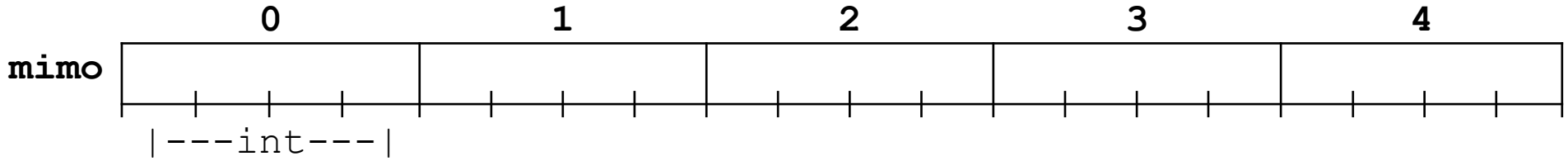
MEMORY MANAGEMENT

- A *variable* is an area of *memory* that has been given a name. For example: the declaration `int x;` acquires four byte memory area and this area of memory that has been given the name `x`.
- One can use the name to specify where to store data. For example: `x=10;` is an instruction to store the data value `10` in the area of memory named `x`.
- The computer access its own memory not by using variable names but by using a memory map with each location of memory uniquely defined by a number, called the *address* of that memory location.
- To access the memory of a variable the program uses the `&` operator. For Example: `&x` returns the *address* of the variable `x`.



ARRAY

- An array can hold a series of elements of the same type placed in contiguous memory locations.
- Each of these elements can be individually referenced by using an index to a unique identifier.
- In other words, arrays are a convenient way of grouping a lot of values of same type under a single variable name.
- For example, an array to contain `5` integer values of type `int` called `mimo` could be represented like this:



where each blank panel represents an element of the array, that in this case are integer values of type `int` with size `4` bytes. These elements are numbered/indexed from `0` to `4` since in arrays the first index is always `0`, independently of its length.

DECLARATION

- Like a regular variable, an array must be declared before it is used. A typical declaration for an array in C++ is:

```
type name [total_number_of_elements];
```

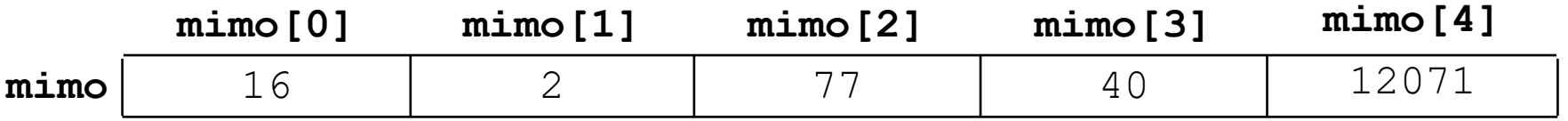
- where `type` is a valid data type (like `int`, `float` …), `name` is a valid identifier and the elements field (which is always enclosed in square brackets `[]`), specifies how many of these elements the array can contain.
- Therefore, in order to declare an array called `mimo` as the one shown in the above diagram it is as simple as:

```
int mimo [5];
```

- The elements field within brackets `[]` which represents the number of elements the array is going to hold, must be a constant integer value, since arrays are blocks of non-dynamic memory whose size must be determined before execution.

INITIALIZATION

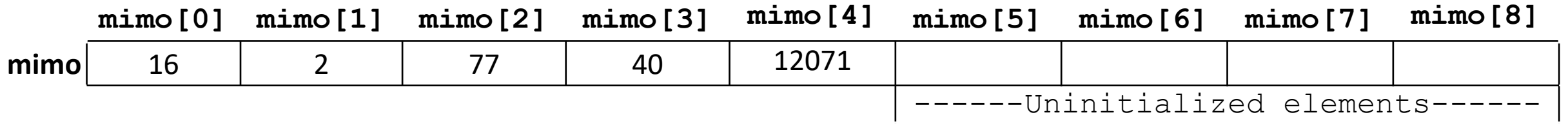
- When declaring a regular array (`int mimo[5];`) of local scope (within a function, for example) its elements will not be initialized to any value by default, so their content will be undetermined until we store some value in them. The elements of global and static arrays are automatically initialized with their default values, zeros.
- When we declare an array, we have the possibility to assign initial values to each one of its elements by enclosing the values in braces `{ }` separated by comas. For example: `int mimo[5] = { 16, 2, 77, 40, 12071 };`
- This declaration would have created an array like this:



- The number of values between braces `{ }` must not be larger than the number of elements that we declare for the array between square brackets `[ ]`.

INITIALIZATION

- An array can also be partially initialized. i.e. we assign values to some of the initial elements. For example: `int mimo[9] = { 16, 2, 77, 40, 12071 };`
- This declaration would have created an array like this:



- Here the first 5 values are assigned sequentially. The rest 4 elements are unassigned.
- Some more initialization –

```
float x[5] = {5.6, 5.7, 5.8, 5.9, 6.1};
char  vowel[6] = {'a', 'e', 'i', 'o', 'u', '\0'};
```

is equivalent to string declaration: `char vowel[6] = "aeiou";`

ACCESS

- The number in the square brackets `[ ]` of the array is referred to as the '*index*' (plural: *indices*) or '*subscript*' of the array and it must be an integer number `0` to *one less than the declared number of elements*.
- We can access the value of any of its elements individually as if it was a normal variable, thus being able to both read and modify its value. The format is as simple as: `name[index]`
- For the declaration `int mimo[5];` the five (`5`) elements in `mimo` is referred in the program by writing: `mimo[0] mimo[1] mimo[2] mimo[3] mimo[4]`
- Each of the above array elements is an integer and each of them also acts as an integer variable. That is, whatever operation we can perform with an integer variable we can do the same with these array elements using same set of rules.

SOME FACTS

- Arrays have a natural partner in programs: the `for` loop. The `for` loop provides a simple way of counting through the numbers of an index in a controlled way. The `for` loop can be used to work on an array sequentially at any time during a program.
- It is important to be able to clearly distinguish between the two uses that brackets `[ ]` have related to arrays.
  - one is to specify the size of arrays when they are declared - `char array[5];` Here the index (`5`) is always an integer constant.
  - the second one is to specify indices for concrete array elements - `array[3] = '*';` Here the index (`3`) is always an integer or an integer expression.

SOME FACTS

- Consider the following example:

```
char array[5];
array[7] = '*';
```

- The assignment of `array[7]` is clearly wrong as index `7` or element `8` in `array[5]` does not exists. But, C/C++ would happily try to write the character `*` at the index `7`. Unfortunately this would probably be written in the memory taken up by some other variable or perhaps even by the operating system. The result would be either:
  - The value in the incorrect memory location would be corrupted with unpredictable consequences.
  - The value would corrupt the memory and crash the program completely! On Unix systems this leads to a memory *segmentation fault*.
- The second of these tends to be the result on operating systems with proper memory protection. Writing over the bounds of an array is a common source of error. Remember that the array limits run from zero to the size of the array minus one.

Array Access Demonstration

```
1.  int mimo[5], a, b, i; // declaration of a new array
2.  mimo[2] = 75;         // store 75 in the third element of mimo
3.  a = mimo[2];          // copy/assign a with the third value of mimo
4.  scanf("%d",&mimo[2]); // read a value for the third element of mimo
5.  // read 5 values for the five elements of mimo sequentially
6.  for(i=0; i<5; i++)
7.     scanf("%d",&mimo[i]);
8.  // print 5 values for the five elements of mimo sequentially
9.  for(i=0; i<5; i++)
10.    printf("%d\n",mimo[i]);
11. // some more interesting accesses
12. a = 4;
13. mimo[2] = a;
14. mimo[a] = 3;
15. b = mimo[a-2] + 2;      //use of expression in index
16. mimo[mimo[a]] = mimo[2] + b;
```

Operation on Array - Insertion

```
1.  int k, i, n=5, mimo[10]={2, 3, 5, 6, 7}; //partial initialization; n=total elements
2.  mimo[n++] = 8;          // insert value 8 at the end of the array; increase n;
3.  // insert value 1 at the beginning of array
4.  for(i=n; i>0; i--)      //shift all the values one index forward. i.e. the value
5.     mimo[i] = mimo[i-1]; //in index 1 goes to 2, 2 goes to 3,…, (n-1)th goes to nth.
6.  mimo[0] = 1; n++;       //1 is inserted at index 1; n increases;
7.  // insert value 4 in the middle (index k=3) of the array
8.  k = 3;
9.  for(i=n; i>k; i--)      //shift all the values one index forward. i.e. the value
10.    mimo[i] = mimo[i-1]; //in index k goes to k+1,…, (n-1)th goes to nth.
11. mimo[k] = 4; n++;       //4 is inserted at index k; n increases;
12. for(i=0; i<n; i++)      //printing all the values in the array after insert
13.    printf("%d ",mimo[i]);
```

OUTPUT:

1 2 3 4 5 6 7 8

i	4

mimo	0	1	2	3	4	5	6	7	8	9
	1	2	3	4	5	6	7	8		

n
8

k
3

Operation on Array - Deletion

```
1.  int k, i, n=8, mimo[10]={1, 2, 3, 4, 5, 6, 7, 8}; //n=total elements.
2.  n--;          // decrease n; last element 8 is no longer part of list.
3.  // delete value 1 from the beginning of array.
4.  n--;          // deleting the value 1 will decrease the total elements n by one.
5.  for(i=0; i<n; i++)   //shift all the values one index backward. The value in index
6.     mimo[i] = mimo[i+1]; //2 goes to 1, 3 goes to 2,…, nth goes to (n-1)th.
7.  k = 2; // delete value 4 from the middle (index k=2) of the array
8.  n--;          // deleting the value 4 will decrease the total elements n by one.
9.  for(i=k; i<n; i++)      //shift all the values one index forward. i.e. the value
10.    mimo[i] = mimo[i-1]; //in index k+1 goes to k,…, nth goes to (n-1)th.
11. for(i=0; i<n; i++)      //printing all the values in the array after insert
12.    printf("%d ",mimo[i]);
```

OUTPUT:

2 3 5 6 7

i	6

mimo	0	1	2	3	4	5	6	7	8	9
	1	2	3	4	5	6	7	8		

n
6

k
2

TWO-DIMENSIONAL ARRAY

- Two-dimensional arrays can be described as "arrays of arrays". For example, a 2D array can be imagined as a Two-dimensional table made of elements of same uniform data type.

minu {

	0	1	2	3	4
0	minu[0][0]	minu[0][1]	minu[0][2]	minu[0][3]	minu[0][4]
1	minu[1][0]	minu[1][1]	minu[1][2]	minu[1][3]	minu[1][4]
2	minu[2][0]	minu[2][1]	minu[2][2]	minu[2][3]	minu[2][4]

- `minu` represents a Two-dimensional array of `3` per `5` elements of type `int`. The way to declare this array in C++ would be: `int minu [3][5];`
- The way to reference the 2nd element vertically and 4th horizontally or the (2 × 4) 8th element in an expression would be: `minu [1][3];`
- Generally, for two-dimensional array, 1st dimension is considered as row and the 2nd dimension is considered as column. Here, we have `3` rows and `5` columns.

INITIALIZATION

- Assigning values at the time of declaring a two-dimensional array can be any one of the following ways:

```
int minu[3][5] = {1,2,3,4,5,2,4,6,8,10,3,6,9,12,15};
int minu[3][5] = {{1,2,3,4,5},{2,4,6,8,10},{3,6,9,12,15}};
int minu[3][5] = {
  {1,2,3,4,5},
  {2,4,6,8,10},
  {3,6,9,12,15}
 };
```

- The internal braces are unnecessary, but helps to distinguish the rows from the columns.
- Take care to include the semicolon at the end of the curly brace which closes the assignment.
- If there are not enough elements in the curly braces to account for every single element in an array, the remaining elements will be filled out with garbage/zeros.
- Static and global variables are always guaranteed to be initialized to zero anyway, whereas auto or local variables are guaranteed to be garbage.

ACCESS

- Nested loop is used to take input and give output.
- The input is taken in row (1st dimension) major. i.e. all the values of row 0 is scanned first, then the values of row 1, and values of row 2. For each row, value at column 0 is scanned first, then the value at column 1, and value at column 2.
- In output, array `a` is used in row major. But array `b` is used in column (2nd dimension) major. i.e. all the values of column 0 is added first, then the values of column 1, and values of column 2. For each column, value at row 0 is added first, then the value at row 1, and value at row 2.

Consider the following example (the dark rea at the end is the output of this program; the red colored text represents input given by the user):

```
1  // input & output of a 2D array
2  void main (void)
3  {
4    int i, j, a[3][3], b[3][3]={1,3,5,7,9,2,4,6,8};
5    for(i=0;i<3;i++)
6      for(j=0;j<3;j++)
7        scanf("%d", &a[i][j]);
8    for(i=0;i<3;i++)
9      for(j=0;j<3;j++)
10       printf("%d ", a[i][j] + b[j][i]);
11 }
```

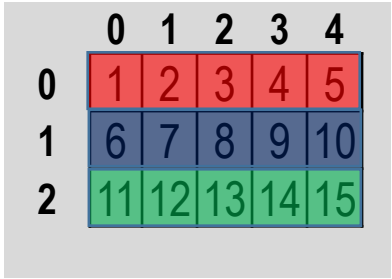
```
2 4 6 8 1 3 5 7 9
3 11 10 11 10 9 10 9 17
```

MEMORY ACCESS

Two-dimensional arrays are just an abstraction for programmers, since we can obtain the same results with a simple array just by putting a factor between its indices: `int minu [3][5];` is equivalent to (3 * 5 = 15); `int minu [15];`

2D Array

```
int minu [3][5], H=3, W=5, n, m, i=0;
void main (void){
  for (n=0; n<H; n++)
    for (m=0; m<W; m++)
      minu [n][m]= ++i;
}
```

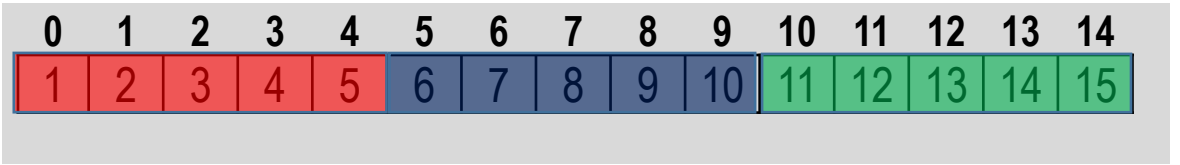


1D array

```
int minu [3 * 5], H=3, W=5, n, m, i=0;
void main (void){
  for (n=0; n<H; n++)
    for (m=0; m<W; m++)
      minu [ W * n + m ] = ++i;
}
```

As memory is flat, in both codes the values are actually stored sequentially in the memory (just like the 1D array). The access for the two-dimensional array in that case is just as the indexing of the array,

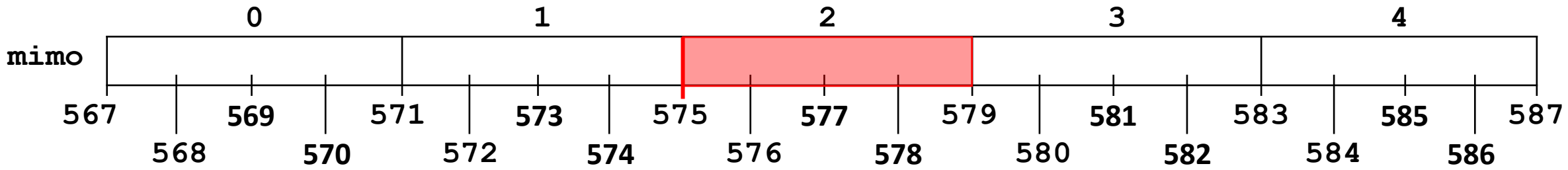
`[(row_index) * (Total_column) + (column_index)]`



Height	Width	n	m	Width * n + m
3	5	0	0	0

MEMORY ACCESS…

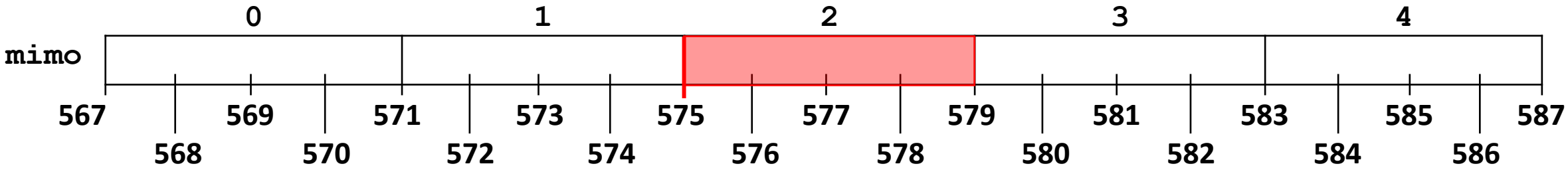
- Memory of each element of an array can be accessed using the **`&`** operator.
- **`&mimo[2]`** gives the memory location of the 3rd element of the array **`mimo`**.
- If the element is more than a byte, it gives the starting byte of the element.
- Let consider the starting address of **`int mimo[5]`** is **`567`**.



- **`&mimo[2]`** will give us the memory location **`575`**.
- **`mimo[2]`** will give us 4 bytes (**`int`**) of information starting from **`575`** to **`579`**.
- The name of an array always refer to the starting location of the array. i.e. the first element of the array. So, **`mimo = &mimo[0]`**.

MEMORY ACCESS…

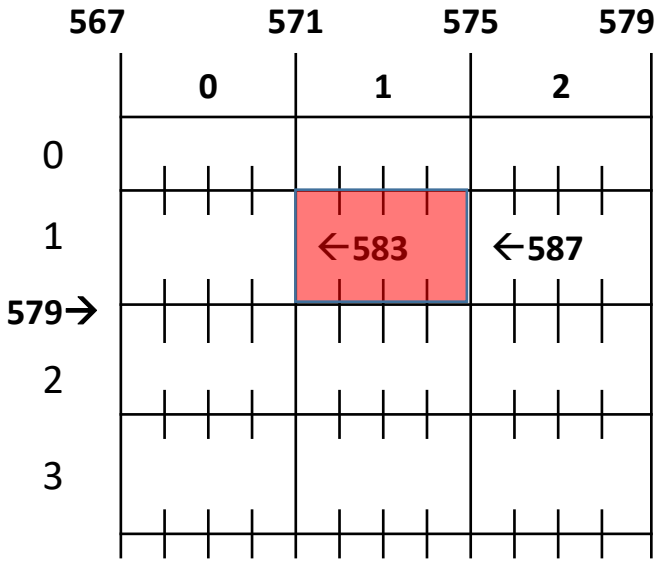
- **`&mimo[2]`** will give us the memory location **`575`**.
- **`mimo[2]`** will give us 4 bytes (**`int`**) of information starting from **`575`** to **`579`**.
- The name of an array always refer to the starting location of the array. i.e. the first element of the array. So, **`mimo = &mimo[0]`**.

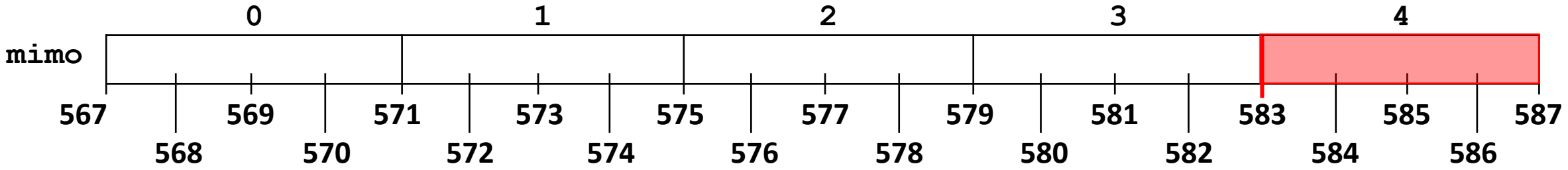



```
&array[index]=start_location_array + index * size_of_data
⇒ &mimo[ 2 ] = mimo (or &mimo[0])  +  2    * sizeof(int)
⇒ &mimo[ 2 ] = 567 + 2 * 4 = 575
```

MEMORY ACCESS...

- Consider a 2D array **mimo[R][C]** each element addressed by **&mimo[i][j]**, where **R**=total element in 1st dimension, **C**=total element in 2nd dimension, **0≤ i < R, 0 ≤ j < C**.
- Let **int mimo[4][3];** Here, **mimo** or **&mimo[0][0]** gives us the starting memory location **567**.
- **mimo[1][1]** will give us 4 bytes (**int**) of information starting from **583** to **587**.





&array[i][j]=start_location + (i * (C * size_of_data)) + (j * size_of_data)

⇒&mimo[1][1] = mimo + (1 * (3 * sizeof(int))) + (1 * sizeof(int))

⇒&mimo[1][1] = 567 + (1 * 3 * 4) + (1 * 4) = 583

MEMORY ACCESS…

- There is a general way to access the memory location of a 2 dimensional array.
- For an array `int mimo[R][C];` and $0 \leq i < R$; $0 \leq j < C$.
  - `mimo[i] = &mimo[0][j]` represents the starting address of ith row.
  - `mimo[i]` skips **`i`** number of rows each with **`C`** number of elements from the **start_location** of the array.
  - So, `mimo[i] = start_location + (i * C elements)`, where `C elements` are counted in bytes based on the **`size_of_data`**, here `int`.
  - So, `mimo[i] = start_location + (i * (C * size_of_data))`.
  - So, `mimo[i] = mimo(or &mimo[0][0]) + (i * (C *sizeof(int)))`.
- A 2D array is also referred as an array of arrays. i.e. an array of which each element is another array.

More than 2 Dimensional Arrays

- Arrays with any number of dimensions by using same number indices/subscripts can be defined `type name[D1][D2][D3]…[Dn]` where `Di`, `1≤i≤n` is the total number of elements in that dimension.
- For an array with `n` dimensions –
  - `name`, the array name always represent the first location/element of the array.
  - The location of any element is accessed by `&name[d1][d2]…[dn]`, where $0 \le d_i < D_i$ is the index of the dimension D_i.
  - `name[d1][d2]…[di-1]`, represents the starting location of the dimension D_i , $1 \le i \le n$.
  - The element of the array is accessed by `name[d1][d2]…[dn]`, where $0 \le d_i < D_i$ is the index of the dimension D_i.
  - `name[d1][d2]…[dn]` represents the jth = $(\sum_{i=1}^{n}(d_i \times \prod_{k=i+1}^{n} D_k) + 1)^{th}$ element of the array.
  - `&name[d1][d2]…[dn]` (= $\sum_{i=1}^{n}(d_i \times \prod_{k=i+1}^{n} D_k) \times size_of_data$) represents the location of jth element of the array.
  - Here, $\sum_{i=1}^{n}(d_i \times \prod_{k=i+1}^{n} D_k)$ represents the index (starting from `0`) of the element, `name[d1][d2]…[dn]` represented in an one dimensional array of [$D_1 \times D_2 \times D_3 \times \dots \times D_n$] elements. The same representation is made in the memory as well.

More than 2 Dimensional Arrays

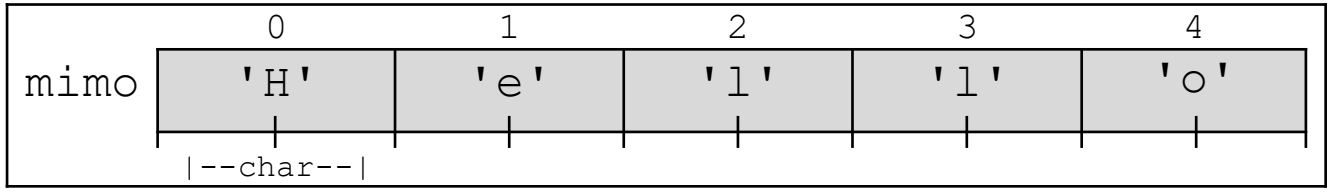
- Consider, `int mimo[4][3][5][2];` Arrays with any number of dimensions by using same number indices/subscripts can be defined `type name[D1][D2][D3]…[Dn]` where D_i, $0 \le i < n$ is the total number of elements in that dimension.
- The memory representation of `mimo[4][3][5][2]` will be the single dimension representation of an array with `[4 × 3 × 5 × 2] = [120]` elements.
- `mimo=&mimo[0][0][0][0]`, represent the first location/element of the array.
  - Accessing an element: `name[2][0][3][1]` represents the 68th $((2\times D_2\times D_3\times D_4)+(0\times D_3\times D_4)+(3\times D_4)+1)=((2\times 3\times 5\times 2)+(0\times 5\times 2)+(3\times 2)+1)+1=68)$ element of the array or memory location.
  - Accessing the location of an element: `&name[2][0][3][1]` $(=((2\times D_2\times D_3\times D_4)+(0\times D_3\times D_4)+(3\times D_4)+1)\times size_of_data))$.

STRING

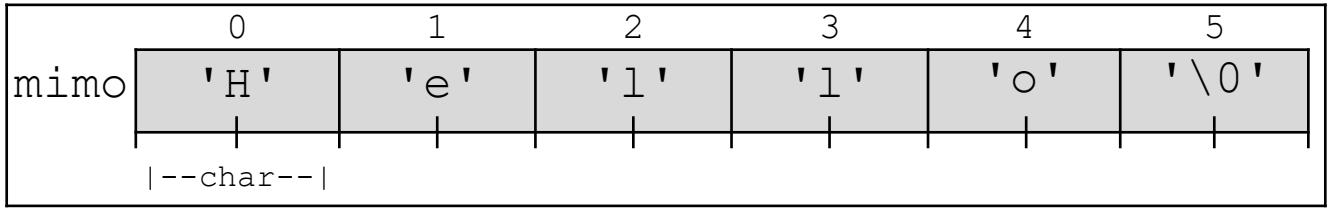
- Strings are sequence of characters representing a piece of text. In programming languages a string is represented as some characters enclosed by double quotes- "`This is a string`".
- A string is mainly declared using an array of characters.
- The main difference between a *simple array of character* and *an array of character representing string* is the end marker given at the end of a string. The standard library functions can recognize this end marker as being the end of the string.
- The end marker is called the zero (or `NULL`) byte because it is just a byte which contains the value zero: '`\0`'. Programs rarely gets to see this end marker as most functions which handle strings use it or add it automatically.

STRING - DECLARATION & INITIALIZATION

- Declaration of a string is just an array of characters: `char mimo [5];`.
- But things are different when we initialize during declaration. An array to contain `5` character values (`'H'`, `'e'`, `'l'`, `'l'`, `'o'`) of type `char` called `mimo` could be represented like this: `char mimo [5]={ 'H','e','l','l','o'};`



- But, to represent the same text as string in C/C++ double quotation (") is used to bound the text. And, a `NULL` character is added at the end of the text. So, because of this `NULL`, we need to declare an additional slot in the array. `char mimo [6]= "Hello";`



- The standard library functions can recognize `NULL` (`'\0'`) as being the end of the string.

String - Access, Input, Output

- Lines 5-7declares three arrays of characters. First two are initialized as strings with a NULL character at their end.
- Line 8 output the string in `Question`. First parameter in `printf` has a `%s` (here `s` for string), and the second parameter is the array name `Question` (giving starting address of the array `Question`).
- The same can be found in line 9, where we have `scanf` with `%s` and the array `FirstName`.
- In all such cases, the `%s` indicates a string, a continuous group of characters, that will be processed one after another.
- The address indicates the location in the memory from where the processing will start.
- The `NULL` character indicates where the processing will stop for `printf`.
- And for `scanf`, after the processing (i.e, after the string input) a `NULL` character is automatically added at the end of the string.

Consider the following example (the dark rea at the end is the output of this program; the red colored text represents input given by the user):

```
1  // null-terminated sequences of characters
2  void main (void)
3  {
4    char Question[]="Please, enter first name: ";
5    char Greeting[] = "Hello";
6    char FirstName[80];
7    printf("%s", Question);
8    scanf("%s", FirstName);
9    printf("%s, %s!", Greeting, FirstName);
10 }
```

```
Please, enter first name: John
Hello, John!
```

STRING - ACCESS, INPUT, OUTPUT...

- Instead of asking for the first name only, if we would have asked for the whole name, this program might not work properly. That is, if we would have given input `John Rambo` instead of only `John` the output will be as follows –

```
Please, enter your first name: John Rambo
Hello, John!
```

- Still it shows `John` after `Hello`, not `John Rambo`. As we know, that `scanf` always stops at `white spaces` during taking input. So, after taking `John` as input `scanf` receives a `space` indicating the end of input. So it never even goes for `Rambo`. To overcome this, there is another function `gets()` which takes only a string as input until a newline ('`\n`') encounters.
- So just writing `gets(FirstName);` instead of `printf("%s", Question);` will give the following output –

```
Please, enter your first name: John Rambo
Hello, John Rambo!
```

- Just like `gets()` for reading from the user, there is also `puts()` which prints the string along with a newline ('`\n`') character at the end of the string.

STRING HANDLING FUNCTIONS

- Strings are often needed to be manipulated by programmer according to the need of a problem. All string manipulation can be done manually by the programmer but, this makes programming complex and large.
- The C/C++ supports a large number of string handling functions. There are numerous functions defined in "`string.h`"/"`cstring`" header file.
- For all such library functions a `NULL` character is assumed to be at the end of the existing string and a `NULL` character is always included at the end of the new/updated/changed string.
- Few commonly used string handling functions are discussed next:

String Handling Functions

Function	Work Of Function
strlen(str)	Calculates the length of string `str`; the total number of characters from the given starting address by `str` until a `NULL` encounters.
strcpy(s1, s2)	Copies a string `str2` to another string `str1`; all the characters starting from the given starting address by `str2` until a `NULL` encounters, will be copied in sequentially from the given address by `str1`.
strcat(s1, s2)	Concatenates (joins) two strings; all the characters starting from the given starting address by `str2` until a `NULL` encounters, will be copied in sequentially from the `NULL` character at the end of `str1`.
strcmp(s1, s2)	Compares two strings; if `str1` and `str2` are equal `strcmp` returns `0`, if `str1` is alphabetically (compares `ASCII` value) lower than `str2` then `strcmp` returns `<0` and otherwise returns `>0`;